

CFD-ML-BASED TURBULENCE MODEL DEFINITION FOR FIDELITY IMPROVEMENT OF HORIZONTAL-AXIS WIND TURBINE RANS SIMULATIONS

Andrea Carloni¹
, Giovanni Delibra²
, Filippo De Girolamo³
, Lorenzo Tieghi⁴
, Alessandro Corsini²
, Valerio Francesco Barnabei²

1. Department of Electrical and Energy Engineering, Sapienza University of Rome, Italy;
email: andrea.carloni@uniroma1.it
2. Department of Mechanical and Aerospace Engineering, Sapienza University of Rome, Italy;
email: giovanni.delibra@uniroma1.it
3. Division of Fluid Mechanics, KTH Royal Institute of Technology, Stockholm, Sweden;
email: fidg@kth.se
4. Department of Civil, Environmental and Mechanical Engineering, University of Trento,
Trento, Italy; email: lorenzo.tieghi@unitn.it

ABSTRACT

Accurate prediction of horizontal-axis wind turbine (HAWT) wakes is essential for wind farm layout optimisation, energy yield estimation and load assessment. Despite that, it remains a major challenge for affordable simulation tools. Large Eddy Simulation (LES) reproduces wake physics reliably but at a prohibitive computational cost, while unsteady Reynolds-Averaged Navier-Stokes (URANS) systematically mis-predicts the wake recovery because of the assumptions embedded in the eddy-viscosity hypothesis. This work proposes a CFD-ML methodology that defines, rather than post-processes, a machine learning-based turbulence closure for HAWT wakes for URANS $k-\omega$ SST models. From a high-fidelity LES of the NREL-5MW turbine operating in a turbulent atmospheric boundary layer (ABL), an LES-equivalent eddy-viscosity field $\nu_{t,eq}$ is extracted and used as the reference target. A K-means clustering algorithm first segments the flow into physically distinct regions, and a Multi-Layer Perceptron (MLP) is then trained to predict the local correction $\Delta \nu_t = \nu_{t,eq} - \nu_{t,URANS}$. The resulting closure is embedded directly into a modified OpenFOAM PIMPLE solver using the $k-\omega$ SST URANS model and evaluated in a fully predictive setting on progressively coarser grids. The trained network reproduces the LES-equivalent eddy viscosity with a validation coefficient of determination $R^2 = 0.9934$, against $R^2 = -7.0$ for the baseline RANS field, and the clustering step is shown to improve the reconstruction in the near-terrain region. Once embedded in the solver, the ML-based RANS model consistently outperforms the standard RANS model in both the near- and far-wake. Most notably, on ultra-coarse grids the model accurately captures the post-breakdown wake recovery at a computational cost about 1.25 times lower than the reference LES, while also reproducing the rotor forces and power coefficients. These results indicate that compact, data-driven turbulence closures can deliver LES-quality wake predictions at a cost compatible with practical wind farm applications.

Keywords: Wind turbine wake, URANS, Machine learning, Turbulence modelling, LES, CFD, Clustering, Neural Networks

NOMENCLATURE

$\vec{U}$	velocity field [m/s]
$\vec{U}_{cyl}$	velocity field in cylindrical coordinates, (U_x, U_θ, U_ρ) [m/s]
U_{ref}	reference inflow velocity at hub height [m/s]
U_θ	tangential velocity component [m/s]
U_ρ	radial velocity component [m/s]
p	kinematic pressure [m ² /s ²]
k	turbulent kinetic energy [m ² /s ²]
D	rotor diameter [m]
z	vertical coordinate from the ground [m]
z_0	aerodynamic roughness length [m]
z_{ref}	reference (hub) height [m]
$x/D, z/D$	streamwise and vertical coordinates normalised by the rotor diameter [-]
S_{ij}	resolved strain-rate tensor [1/s]
R_{ij}^{res}	resolved Reynolds stress tensor [m ² /s ²]
Q	second invariant of the velocity gradient tensor (Q-criterion) [1/s ²]
$\mathbf{R}$	Reynolds stress tensor [m ² /s ²]
C_p, C_t	power and thrust coefficients [-]
ϵ	Gaussian projection width of the ALM body force [m]
Δx	cell size [m]
Δb	actuator-line element spacing [m]
Δt	simulation time step [s]
R^2	coefficient of determination [-]
Inertia	within-cluster sum of squares (WCSS) minimised by K-means [-]
K	number of clusters [-]
C_i, μ_i	set of points and centroid of cluster i [-]
w_{ax}	axial sample weight in the NN loss [-]
n_0, n_i, r	initial neuron width, width of layer i , width-decay factor [-]
ν	molecular kinematic viscosity [m ² /s]
ν_{eff}	effective kinematic viscosity, $\nu + \nu_t$ [m ² /s]
ν_t	turbulent (eddy) kinematic viscosity [m ² /s]
$\nu_{t,RANS}$	baseline RANS eddy viscosity [m ² /s]
$\nu_{t,eq}$	LES-equivalent eddy viscosity (regression target reference) [m ² /s]
$\nu_{t,mod}$	modified eddy viscosity reproducing the LES mean fields [m ² /s]
$\nu_{t,ML}$	ML-reconstructed eddy viscosity, $\nu_{t,RANS} + \Delta \nu_{t,ML}$ [m ² /s]
ν_{sgs}	subgrid-scale eddy viscosity [m ² /s]
$\Delta \nu_t$	eddy-viscosity correction, $\nu_{t,eq} - \nu_{t,RANS}$ [m ² /s]
τ_{ij}^{sgs}	subgrid-scale stress tensor [m ² /s ²]
Σ_{ij}^{dev}	deviatoric part of the total turbulent stress tensor [m ² /s ²]
ω	specific turbulence dissipation rate [1/s]
δ	Huber-loss transition parameter [-]

Acronyms

ABL	Atmospheric Boundary Layer
ALM	Actuator Line Model
CA	Clustering Algorithm

CFD Computational Fluid Dynamics
FIML Field Inversion and Machine Learning
GELU Gaussian Error Linear Unit
HAWT Horizontal-Axis Wind Turbine
LES Large Eddy Simulation
MAE Mean Absolute Error
ML Machine Learning
MLP Multi-Layer Perceptron
MSE Mean Squared Error
NN Neural Network
NREL National Renewable Energy Laboratory
PCA Principal Component Analysis
PC Principal Component
RANS Reynolds-Averaged Navier-Stokes
RMSE Root Mean Squared Error
SCADA Supervisory Control And Data Acquisition
SGS Subgrid-Scale
SST Shear-Stress Transport
WALE Wall-Adapting Local Eddy-viscosity
WCSS Within-Cluster Sum of Squares
WT Wind Turbine

1 INTRODUCTION

Wind energy is one of the cornerstones of the ongoing transition toward sustainable electricity generation. As wind turbines are deployed in increasingly large farms, both onshore and offshore, the aerodynamic interaction among turbines becomes a critical design and operational challenge. When a turbine extracts kinetic energy from the incoming flow, it leaves behind a wake region characterised by a significant velocity deficit and elevated turbulence levels. Downstream turbines that operate within this region consequently suffer both reduced power output and increased fatigue loads, affecting the economic and structural performance of the entire farm [1]. Accurate prediction of wind turbine wake behaviour is therefore essential for wind farm layout optimisation, energy yield estimation, and structural load assessment.

From a fluid-mechanics perspective, the wake of a horizontal-axis wind turbine (HAWT) is a complex, multi-scale phenomenon. It is deeply coupled with the Atmospheric Boundary Layer (ABL), which is itself inherently unsteady and stratified [1]. The near-wake region, dominated by the tip vortex system shed by the rotor blades, transitions into a far-wake where shear-driven turbulent mixing progressively restores the velocity deficit. This recovery process is strongly influenced by the ambient turbulence intensity, wind shear, thermal stability, and the topography of the terrain [2]. The wide range of length and time scales involved, from blade boundary layers to the full extent of the ABL, makes any single simulation approach inherently challenging.

Several modelling strategies are available, each representing a different trade-off between physical fidelity and computational cost. On one end of the spectrum, analytical engineering wake models (e.g., the Jensen top-hat model and its Gaussian successors) offer near-instantaneous evaluation and are therefore routinely used in wind farm layout optimisation [3], but they rely on strong simplifying assumptions and cannot capture the complex turbulent structure of real wakes. On the other end, Large Eddy Simulation (LES) resolves the dominant turbulent structures explicitly and is considered the reference standard for wake physics; however, a single sim-

ulation of a medium-sized wind farm can demand between 10^5 and 3×10^6 CPU hours [4], making it prohibitively expensive for systematic design or optimisation studies. Unsteady Reynolds-Averaged Navier-Stokes (URANS) methods occupy an intermediate position, reducing computational cost by roughly two orders of magnitude relative to LES [4]. Yet the closure assumptions embedded in standard RANS turbulence models, most notably the Boussinesq eddy-viscosity hypothesis and the isotropic treatment of the Reynolds stress tensor, introduce structural model-form errors that are particularly pronounced in separated and wake flows, leading to a systematic mis-prediction of the wake recovery rate [5, 6].

Machine Learning (ML) has emerged as a powerful paradigm to complement or enhance physics-based simulations across several domains of wind energy research. A large body of literature has applied ML to operational tasks such as wind power forecasting from SCADA data [7, 6], predictive maintenance and condition monitoring of turbine components [7], and wind farm layout optimisation using data-driven surrogate models of wake interactions [3, 8]. In these applications ML is generally used downstream of a physical model: a CFD or analytical solver first generates the flow field, and ML then post-processes or surrogate-replaces its output to predict power, loads, or optimal configurations.

A conceptually distinct and more recent line of research aims instead to use ML to correct or replace the turbulence closure itself, modifying the governing equations during the simulation. An example of that can be seen in [9] with the proposed FIML (field inversion and machine learning) proposed by Parish & Duraisamy and the invariant neural-network closure of Ling et al. [10]. In the wind energy context, Steiner et al. [4] demonstrated that optimal corrective fields for the turbulence anisotropy tensor and turbulent kinetic energy production can be derived from time-averaged LES data and used to construct a custom RANS closure via sparse symbolic regression, yielding substantially improved wake predictions for multi-turbine configurations at wind-tunnel scale. Similarly, data-driven Gaussian-process corrections to the eddy-viscosity field, trained on single-turbine LES, have shown improved accuracy when deployed in a wind-plant RANS simulation [11]. More broadly, Steiner [5] showed that combined baseline-RANS plus data-driven correction strategies can outperform standard models for both velocity and turbulent kinetic energy, while acknowledging that larger datasets and more stable numerical implementations are needed before such methods can be applied at industrial scale.

The present work contributes to this emerging research direction, with specific novelty in three respects. First, rather than identifying a global correction to the turbulence model, a clustering algorithm (CA) based on the K-means method is employed to segment the flow domain into physically distinct regions characterised by different wake dynamics, following the decomposition approach introduced in [12, 13]. This region-aware strategy naturally improves the generalisability and interpretability of the trained models and makes the correction less sensitive to mesh refinement. Second, a Multi-Layer Perceptron (MLP) is trained to predict the local correction $\Delta \nu_t = \nu_{t,eq} - \nu_{t,RANS}$ between the LES-equivalent eddy viscosity and the RANS baseline, and the resulting data-driven closure is directly embedded into the OpenFOAM PIMPLE solver via a modified turbulence correction loop, enabling its use in fully predictive RANS simulations. Third, the methodology is tested across multiple mesh refinement levels, demonstrating that LES-quality accuracy in wake recovery can be achieved at a computational cost lower than that of standard LES, which is a prerequisite for practical wind farm applications.

It is worth emphasising what distinguishes this approach from the bulk of ML applications in wind energy. Most existing methods employ ML as a post-processing or surrogate tool applied to the output of a turbulence model; the turbulence model itself is left unchanged. Here, the ML component is used to define the turbulence model, to construct, from scratch, a problem-specific eddy-viscosity closure calibrated to reproduce high-fidelity LES statistics. This opens

a path towards compact, computationally efficient flow models that could in the future support real-time wind farm monitoring and control, by providing accurate yet computational effective representations of the wake field and its evolution under varying inflow conditions.

The paper is structured as follows. Sec. 2 describes the CFD campaign used to generate the training and testing data, the clustering pipeline, and the MLP architecture and training strategy. Sec. 3 introduces the NREL-5MW reference turbine case study and the computational domain. Results are presented in Sec. 4, covering the clustering, the neural network training metrics, and the performance of the ML-based RANS model on coarser grids. Conclusions are drawn in Sec. 5.

2 METHODOLOGY

The methodology adopted relies both, as cited, on a CFD and on a ML part. The CFD part is used to generate the data and test the model while the ML part is used to train the models, both Clustering and NN.

2.1 CFD dataset

A CFD simulation campaign is performed to generate the data for the ML part. Simulations are performed via OpenFOAM-v22.12, an open-source CFD software including an in-house implementation of Actuator Line Model (ALM)[14] based on the open-source library *turbines-Foam*[15]. Both RANS and LES simulations are performed, assuming incompressible flow conditions. $\vec{U}$ and p fields are solved with the PIMPLE algorithm. In all the simulations the timestep, corresponding to 2° of rotation of the WT rotor, is set to ensure a maximum Courant number of 0.6 with a mean value of 0.02-0.03, while the time integration relied on a first-order Euler scheme. Second order central differences were used for the spatial discretisation of the convective and diffusive terms.

The RANS simulations are performed with the $k-\omega$ SST turbulence model [16] while the LES simulations are performed with the WALE turbulence model [17].

The following boundary conditions were applied in the RANS case: at the inlet a logarithmic velocity profile was imposed. At the outlet, a zero-gradient condition was set for the $\vec{U}$ field. No-slip boundary conditions were set for the ground, while freestream conditions were applied to the other lateral boundaries. For pressure a zero-gradient condition was set for all the boundaries.

In the LES case, the same boundary conditions were applied except for the inlet $\vec{U}$ field, where turbulent fluctuations, derived from the Kaimal spectrum, were superimposed to the logarithmic velocity profile. Synthetic inflow conditions based on a spectra-stochastic method are imposed at the inlet derived from the Kaimal power spectral density function [18] following the approach described in [19]. As done in [13], the turbulent fluctuations are generated by a coded boundary condition, imposing a total of 400 Fourier modes. Continuity is ensured by constraining the orthogonality of the wave numbers and $\vec{U}$. Spatial coherence is imposed through an exponential decay function $\exp(-\frac{\Delta t}{T})$ where Δt is the time step of the simulations and T is the time scale of the turbulence considered.

Moreover, to generate the data for the ML regression, a URANS simulation was executed importing the $\vec{U}$ and p mean fields from a LES simulation in it. Then a single timestep was executed, fixing the $\vec{U}$ and p fields to the imported ones, to compute the modified eddy viscosity field $\nu_{t,mod}$. To do so, a modified version of the SIMPLE algorithm was implemented, where the ν_t field is solved accordingly to the LES input fields inside the turbulence correction loop.

All the model training and testing is performed on the same computational domain, which is a rectangular box where the rotor is placed at 5D from the inlet and 15D from the outlet and 90m from terrain. In spanwise direction the domain extends for 5D on each side of the rotor and

in normal direction $8D$ from the ground. The domain is obtained by using the *blockMesh* and *snappyHexMesh* utilities of OpenFOAM, with different refinement levels in different regions of the domain to proper solve the turbulent structures in the near wake of the rotor. Moreover, an inflation layer is applied near terrain. The final mesh consists of around ~ 30.8 million cells with a minimum cell size of 1.75 m near the rotor and a maximum cell size of 28 m at the domain boundaries while reaching ~ 0.33 m in the first level of the inflation layer near the ground. In Fig. 1 the computational domain and the mesh refinement are shown.

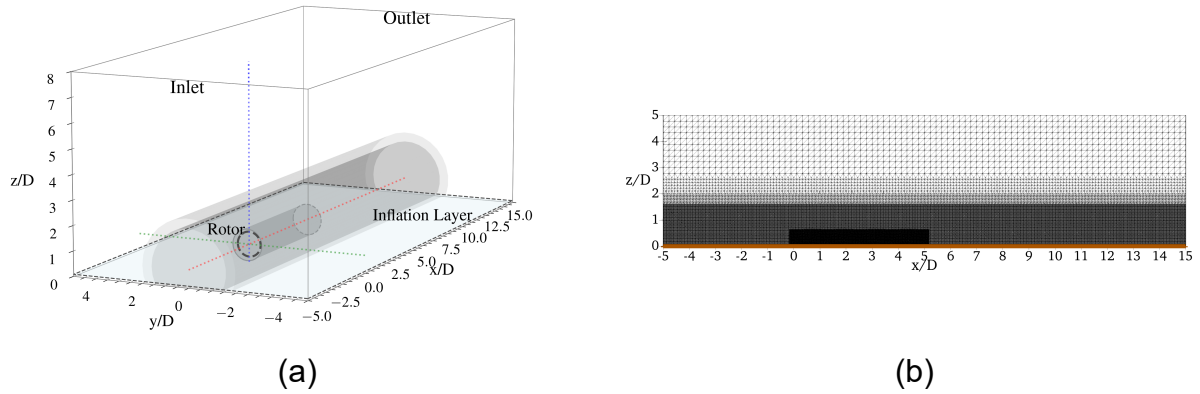


Figure 1. Computational domain (a) and refinement levels represented in (b).

Moreover, the same domain has been used to test the trained models and the mesh refinement influence on the results: while the computational domain and the refinement levels are the same, the minimum and maximum cell size are varied to generate coarser meshes, leading to ~ 15.9 , ~ 9.1 and ~ 4.2 million cells with a minimum cell size of ~ 2.25 m, ~ 2.82 m and ~ 3.94 m respectively near the rotor and a $\times 16$ cell size at the domain boundaries.

The CFD simulations were run on a high-performance computing workstation equipped with two AMD EPYC 9534 64-Core processors, for a total of 128 physical cores and 256 threads.

The three types of simulations (RANS, LES and modified RANS) results are shown in Fig. 2.

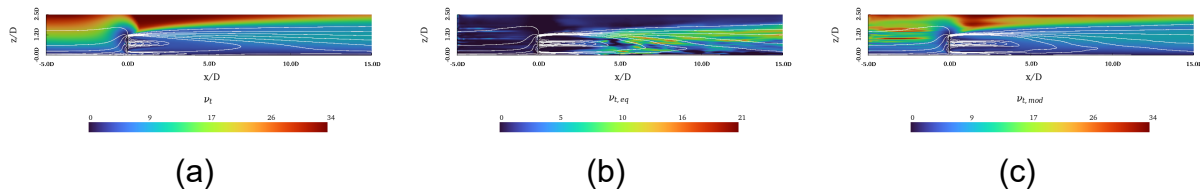


Figure 2. Eddy viscosity fields on the xz plane: (a) original RANS ν_t , (b) LES-equivalent $\nu_{t,eq}$, and (c) modified RANS $\nu_{t,mod}$. The black and white isolines highlight the velocity magnitude contours for RANS simulation and mean velocity ones for the LES and modified RANS simulations.

2.2 ML methodologies

Different ML techniques are employed in this study to predict the modified eddy viscosity field ν_t and the corresponding $\vec{U}$, p and k fields. These methods include both CA and NN, as described

in the following.

All the ML models were trained on the same workstation, equipped with an Intel Core i9-10980XE CPU running at 3.00 GHz, with 18 physical cores and 36 threads, and a single NVIDIA Quadro RTX 6000 GPU with 24 GB of dedicated memory.

Clustering model

This work entails the use of a CA as a cornerstone of all the analysis. This, as in [13, 12], is done to identify the different flow regions in the wake of the rotor, which are characterised by different dynamics and thus require different modeling approaches. Moreover, the usage of the CA has an advantage in terms of interpretability of the results and in terms of generalisation capabilities of the trained models: in fact it allows to move from a cell-wise prediction of the ν_t field to a region-wise prediction, which is more robust and less affected by the mesh refinement and by the geometrical details of the domain.

In fact, clustering algorithms are unsupervised ML techniques that allow to group data points in clusters based on their similarity. Many different CAs exist in literature and with different characteristics, but in this work the K-means algorithm is used as in [13, 12] for many reasons: it is proven to be effective in identifying the different flow regions in the wake of the rotor and its computational complexity scales well with the size of the dataset, which is a crucial aspect in this study given the large amount of data generated by the CFD simulations. The clustering pipeline was implemented in a Python environment leveraging the *scikit-learn* [20] open-source library. The K-means algorithm partitions the dataset by minimising the within-cluster sum of squares, the quantity reported by *scikit-learn* as the *inertia* of the clustering and defined as:

$$\text{Inertia} = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2, \quad (1)$$

where K is the number of clusters, C_i is the set of data points in cluster i , x_j is a data point belonging to cluster i and μ_i is its centroid. The algorithm iteratively assigns each data point to the nearest centroid and updates the centroids until convergence is reached. To select the optimal number of clusters, the elbow method is used, analysing the inertia as a function of the number of clusters K : the chosen K corresponds to the point beyond which additional clusters yield only a marginal reduction of the inertia. To ensure a correct clustering of the data, the features are previously normalised via a Min-Max normalisation [20],

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \quad (2)$$

Then, as done in [13, 12], the features are projected on a cylindrical reference system centered on the rotor and a subset of them is selected based on the PCA analysis and on the physical understanding of the problem, to ensure a correct clustering of the data both in terms of data analysis and physical interpretation of the results. Since the exact pipeline has been already described in previous works [13, 12], here only the main results are presented (i.e., the optimal number of clusters and clustering features). Results of the clustering methodology are reported in Sec. 4.1.

A brief review of the clustering pipeline together with the CFD one is reported in Fig. 3.

NN model

Finally, after the clustering of the data, a NN is trained to predict the $\nu_{t,ML}$ field. In this work, ν_t denotes the kinematic turbulent viscosity, namely the scalar field used in the Boussinesq approximation to model the effect of the unresolved turbulent stresses on the mean momentum

This formulation allows the network to learn the correction required to move from the baseline RANS eddy viscosity to the LES-equivalent eddy viscosity used as target closure.

The regression model is a Multi-Layer Perceptron (MLP), implemented in TensorFlow-Keras [21], with a single scalar output corresponding to $\Delta \nu_{t,ML}$. The input vector is defined at cell level and includes $\vec{U}_{cyl}$, k , $\nu_{t,RANS}$, the distance from the assigned cluster centroid in the normalised clustering space, the non-dimensional vertical and axial coordinates z/D and x/D , and the cluster labels. The continuous features are standardised using a Standard Scaler, while the cluster labels are encoded through one-hot encoding. The target $\Delta \nu_t$ is also standardised before training.

Prior to the split, cells in the far-upstream inflow region ($x/D < -2.5$) are excluded from the training and validation sets, as the NN performance in that zone is inherently limited by the synthetic inlet boundary condition; this reduces the dataset from $\sim 180k$ cells to $\sim 160k$ cells retained for training and validation. The remaining points are split into training and validation subsets using an 80/20 split, stratified with respect to the cluster labels in order to preserve the same distribution of flow regions in both sets. During training, sample weights are introduced to compensate for cluster imbalance through inverse-frequency weighting. An additional axial weight is applied to increase the relevance of downstream wake samples, according to

$$w_{ax} = 1 + \max(x/D, 0), \quad (10)$$

and the final sample weight is obtained as the product of the cluster and axial weights.

The MLP architecture is composed of a sequence of fully connected residual blocks. Each block contains a Dense layer with L2 kernel regularization, Layer Normalization, a GELU activation function and Dropout. When residual connections are enabled, the block input is added to the block output; if the two tensors have different dimensions, the shortcut is first projected through a linear Dense layer. The hidden-layer widths are progressively reduced according to

$$n_i = \max(\text{round}(n_0 r^{i-1}), 32), \quad (11)$$

where n_0 is the initial number of neurons and r is a width-decay factor. This is done in order to reduce the complexity of the model and prevent overfitting and also to catch more complex relationships between the input features in the first layers and to produce a more compact representation. The output layer is a linear Dense layer with one neuron. The model is trained by minimising a weighted Huber loss using the Adam optimizer, while the mean absolute error is monitored as an additional metric.

The architecture and training hyperparameters are fixed through a manual preset, selected from preliminary tests and kept unchanged for the reported training. The adopted configuration is reported in Tab. 1.

The final model is trained with early stopping on the validation loss and a learning-rate reduction on plateau. The maximum number of epochs is controlled by the training configuration and was set to 400 in the reported run. In addition to the training and validation loss, the RMSE and physical-space validation metrics are monitored during training.

Results of the NN training and testing are reported in Sec. 4.2 as for the clustering ones. MLP training pipeline is summarised in Fig. 4.

2.3 ML-based RANS model implementation

Finally all the trained models are implemented in the RANS solver of OpenFOAM to test their performance in a predictive setting. To do so, a modified version of the PIMPLE algorithm is implemented as in Fig. 5. The $\vec{U}_{cyl-p}$ PIMPLE internal loop is not modified, while the turbulence

Table 1. Manual configuration adopted for the MLP training.

Hyperparameter	Value
Hidden blocks	7
Initial neurons n_0	192
Width decay r	0.78
Hidden-layer widths	192, 150, 117, 91, 71, 55, 43
Activation function	GELU
Layer Normalization	True
Residual connections	True
Dropout rate	0.05
L2 regularization	$1 \cdot 10^{-4}$
Learning rate	$1.5 \cdot 10^{-4}$
Huber δ	1.0
Batch size	128

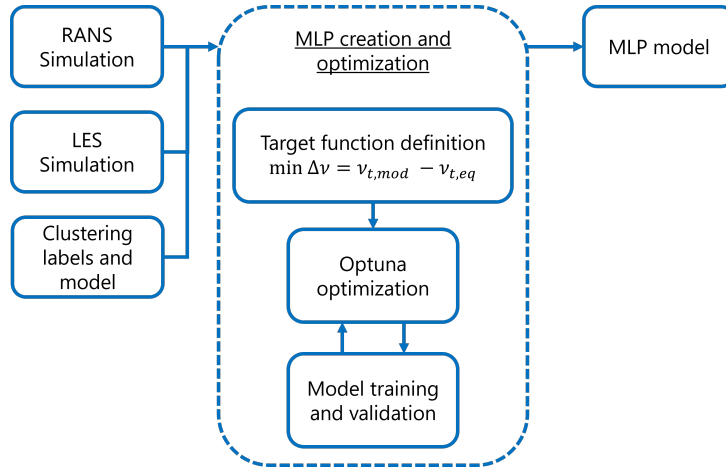


Figure 4. MLP training pipeline.

correction loop is: the first iteration of the solver is executed with the standard k - ω SST model to compute the $\vec{U}$, p , k , ω and v_t fields, then the clustering model is applied to assign a cluster label to each cell and the NN is applied to predict the $v_{t,ML}$ field; this is then used, in the second iteration of the solver, to compute the $\vec{U}$ and p fields, while the k and ω fields are still computed with the standard SST model stored in the first iteration and then corrected again with the NN; the same procedure is repeated for the following iterations of the solver.

3 CASE STUDY

To test the proposed methodology, this study considers the NREL-5MW [22] Horizontal-Axis Wind Turbine (HAWT). As cited before, the rotor is modeled with the ALM to reduce the computational cost of the simulations. Number of nodes are changed according to the mesh refinement, to ensure always that $\epsilon \leq 2\Delta x$ and $\frac{\Delta b}{\Delta x} \leq 0.75$ [23]. Selected combinations of Δb are reported in Tab. 2 as a function of the mesh refinement. Finally, an inflow velocity U_{ref} at hub height (i.e., 90 m from the ground) of 8 m/s is considered, and the ABL logarithmic profile is

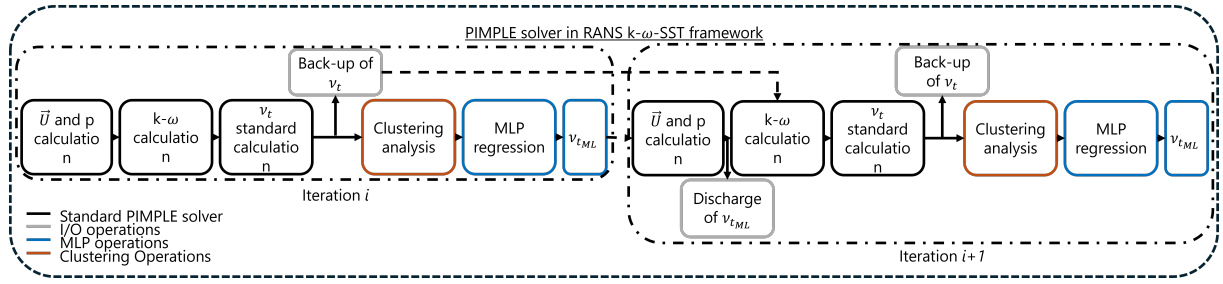


Figure 5. ML-based PIMPLE algorithm.

Mesh Δx [m]	Δb [m]	$\frac{\Delta b}{\Delta x}$
~ 3.94	32	~ 0.5
~ 2.82	44	~ 0.5
~ 2.25	56	~ 0.5
~ 1.78	71	~ 0.5

Table 2. ALM parameters for the different mesh refinements.

evaluated from this condition as reported in Eq. 12:

$$U(z) = U_{ref} \frac{\ln(\frac{z+z_0}{z_0})}{\ln(\frac{z_{ref}+z_0}{z_0})} \quad (12)$$

The data generated with the CFD campaign was evaluated against literature results [24, 25] as also done in previous studies (e.g., [13]). Hence, the results are not reported here for brevity, but the interested reader can refer to the cited works for a detailed validation of the CFD dataset. Such dataset is then used to extract the training data for the ML models and to test them. The training dataset is generated by sampling the CFD fields along two different planes crossing the rotor center from the finer grid: one that is parallel to the terrain and one that is perpendicular to it to catch correctly the wake-ABL induced dynamics. This results in a total of $\sim 180k$ cells for the training dataset as cited in Sec. 2.2.

4 RESULTS

Finally the results are presented. Must be noted that the ML results are referred to the finest grid, which is the one used for training. The ML-based RANS model results are referred to the coarser grid instead to stress the improvement of the ML-based RANS model with respect to the standard RANS model not only in terms of accuracy but also in terms of computational cost.

4.1 Clustering results

First of all the PCA results are shown in Fig. 6a. From Fig. 6a it can be seen that the first 6 PCs explain more than 90% of the variance. From Fig. 6b the influence of the features on the first 6 PCs can be seen. It can be noted that the most influential features are the ones related to the cells coordinates (expressed in cylindrical coordinates) followed by k , v_t and U_x . Q and $\mathbf{R}$ are not considered since they are inherently related to $\vec{U}_{cyl}$ and k so they do not add any information to the clustering. This leads to the choice of U_x , U_θ , U_ρ , k , v_t and the cells coordinates as features for the clustering. Since the spatial information is contained in the clusters, the cell-coordinate-related features are discarded. Moreover, since the clustering operations are performed in a pipeline in which the correction must affect the velocity field, both the k and v_t features are discarded since they are directly related to the correction to be predicted by the NN model,

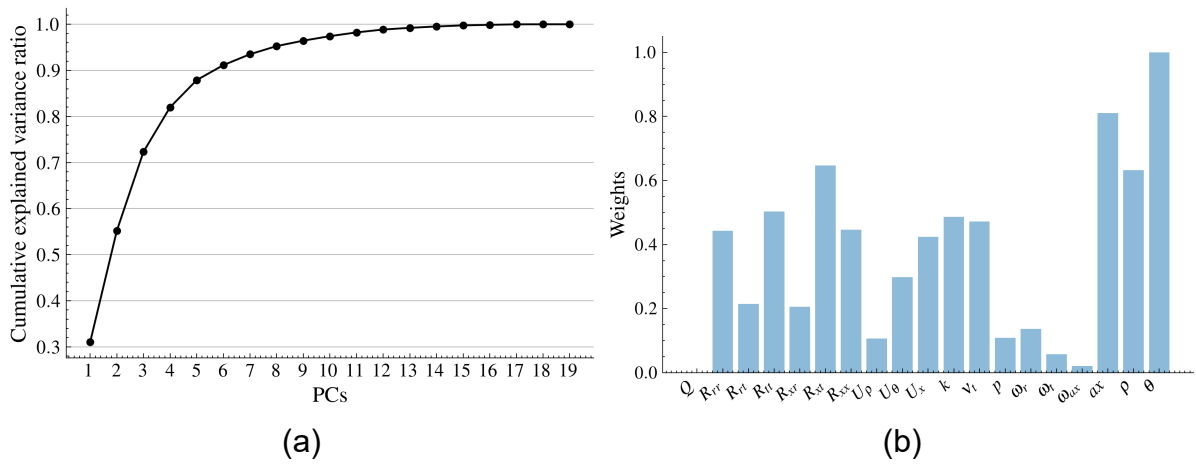


Figure 6. PCA results: (a) explained variance ratio, (b) features weights on the first 6 PCA components.

which is the eddy viscosity correction. Thus, only U_x , U_θ and U_ρ are retained as features for the clustering.

The next step consists into determining the optimal number of clusters. The elbow chart for the clustering is shown in Fig. 7. The plot shows a clear elbow near $K = 7$, which is thus chosen

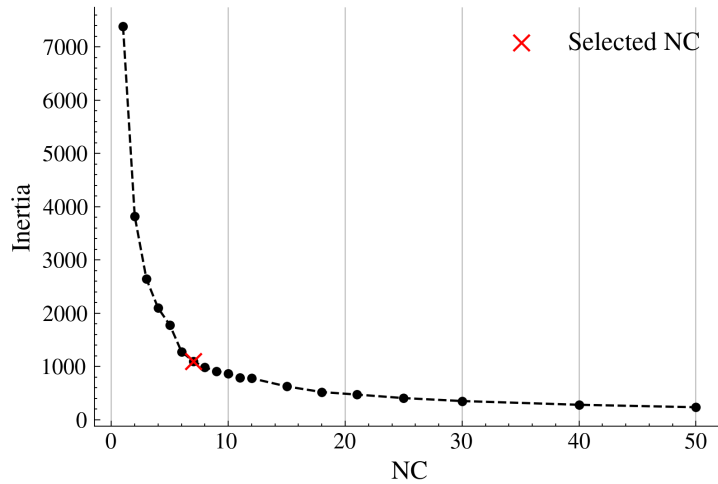


Figure 7. Elbow chart for the clustering.

as the optimal number of clusters. Once all the parameters of the clustering are defined, the clustering is applied to the whole dataset and the results are shown in Fig. 8. Different regions of the flow are clearly identified by the clustering. A clear separation between the disturbed and undisturbed flow can be seen:

- Cluster 1 identifies the undisturbed flow.
- Clusters from 2 to 7 identify wake. In particular, cluster 3 identifies the inlet-ABL-affected region upstream of the rotor and the upper far-wake region; cluster 4 identifies the lower ABL-influenced far-wake region; the near-wake is identified by cluster 6, while cluster 5 highlights the near-wake region with higher k levels and the mixing/breakdown region; finally, cluster 7 identifies the upstream rotor-inlet region.

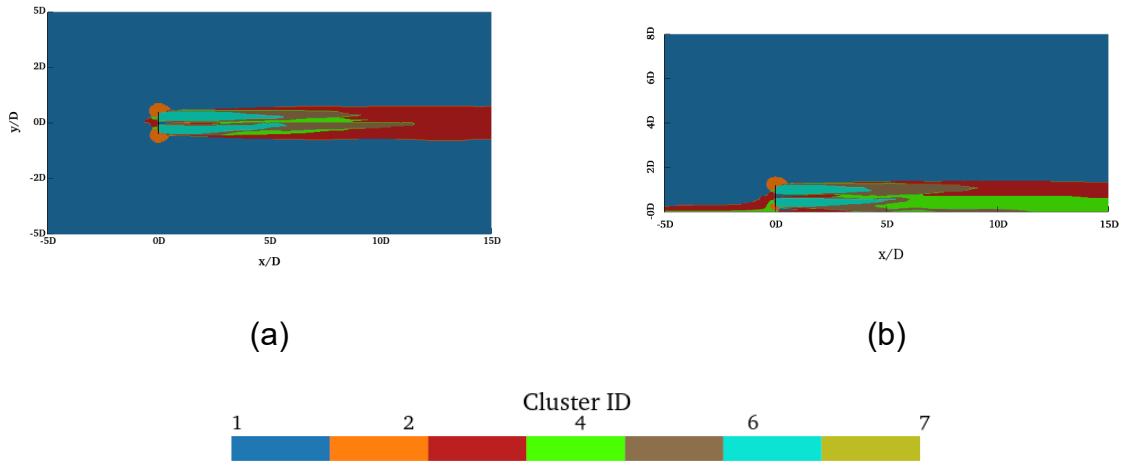


Figure 8. Clustering results: (a) xy view, (b) xz view.

Clusters validation is performed as in [13] and will not be shown here for brevity but the results are consistent with the reference and with the physical phenomena occurring in the flow. The clustering results are thus considered valid and the clusters are used for the training of the ML-based RANS model. Clustering model is trained in ~ 0.24 seconds on the previously defined workstation and dataset.

4.2 NN results

The NN model is trained using the fixed manual configuration described in Sec. 2.2.

The network predicts the standardised correction $\Delta v_t = v_{t,eq} - v_{t,RANS}$ and the final eddy viscosity is reconstructed as $v_{t,ML} = v_{t,RANS} + \Delta v_{t,ML}$. In the reported run, early stopping is triggered at epoch 201 with the best checkpoint at epoch 187, where *validation loss* = $2.96 \cdot 10^{-3}$ and *validation MAE* = $1.99 \cdot 10^{-2}$ in normalized target space. On the training set the reconstructed physical field reaches $R^2 = 0.9935$, $RMSE = 0.2546$ and $MAE = 0.1789$, while the validation subset gives $R^2 = 0.9934$, $RMSE = 0.2588$ and $MAE = 0.1831$.

The training and validation loss curves are shown in Fig. 9 as also the scatter plot of the predicted and true v_t values on the test set. From Fig. 9a it can be seen that the training and validation loss curves show a slight overfitting, coherent with the fact that the dataset is relatively small: in fact only one operating condition is used for training and leveraging a larger dataset would likely lead to better generalisation capabilities. However, the two curves do not diverge significantly and the NN predictions are accurate, as shown in Fig. 9b, where the predicted $\Delta v_{t,ML}$ values are close to the true Δv_t values as also happens in Fig. 9c, where the predicted $v_{t,ML}$ values are close to the true $v_{t,eq}$ values. A series of outliers can be noted in the predictions, in the marked region of the scatter plots, where the NN underestimates the true $v_{t,eq}$ values. This is inherited from the same bias area highlighted in the scatter plot of the $\Delta v_{t,ML}$ values, where the NN underestimates the true Δv_t values. This, as Fig. 9d shows, corresponds to a specific region of the domain, located at the inlet, in which numerical instabilities are present due to the coded inlet boundary condition. In such area the NN predictions are less accurate, but this is not expected to significantly affect the overall performance of the ML-based RANS model. In fact, the NN predictions are still accurate and the NN-based RANS model is expected to perform better than the standard RANS model, as shown in the next section. The resulting $v_{t,ML}$ field is

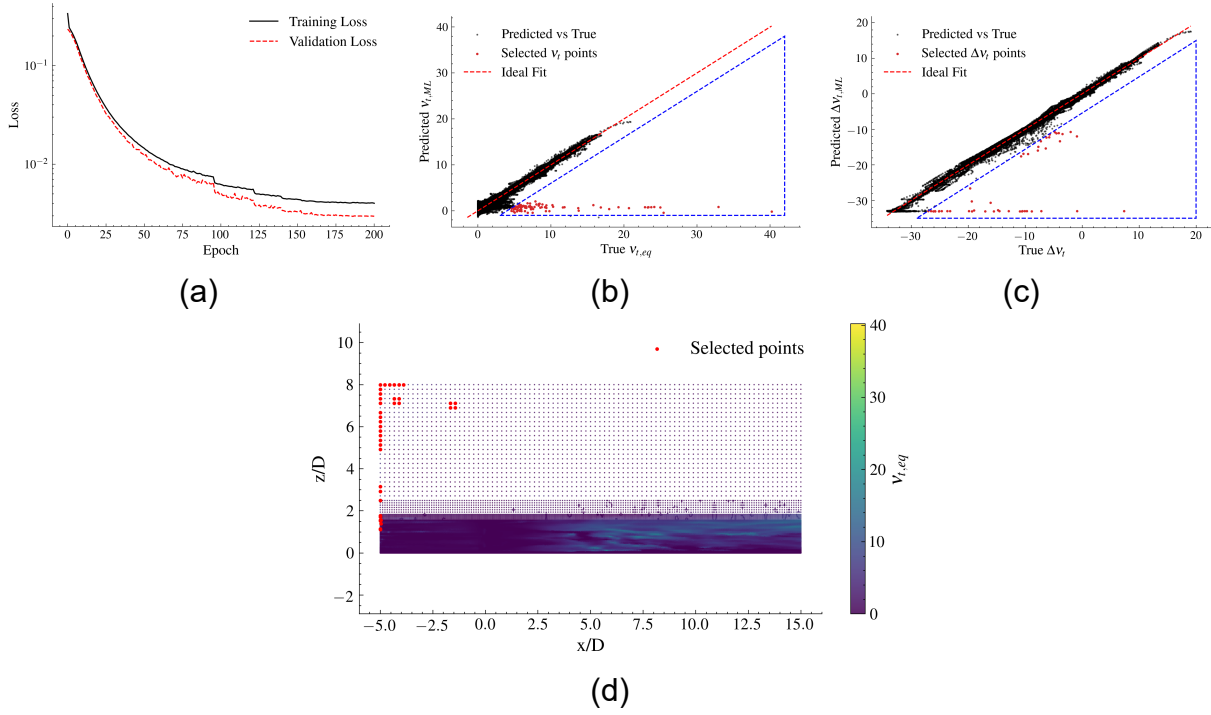


Figure 9. NN results: (a) training and validation loss curves, (b) scatter plot of predicted vs true v_t values on the test set, (c) scatter plot of predicted vs true Δv_t values on the test set, (d) slice plot of selected high-error points.

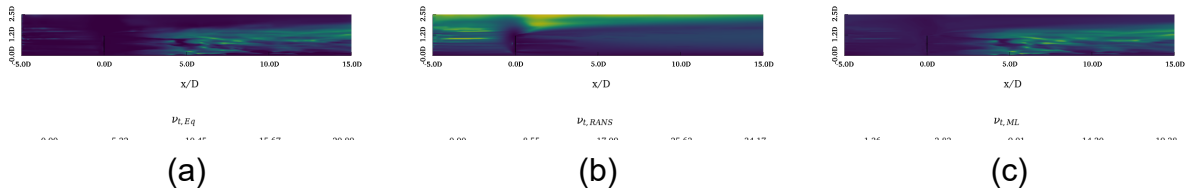


Figure 10. Eddy viscosity fields: (a) $v_{t,eq}$, (b) $v_{t,RANS}$ (obtained by importing the mean pressure and velocity field of the LES in the RANS solver), (c) $v_{t,ML}$.

shown in Fig. 10 together with the $v_{t,eq}$ and $v_{t,RANS}$ fields for comparison.

4.3 ML-based RANS model results

The trained ML model is embedded in the RANS solver as described in Sec. 2.2 and evaluated across grids of different resolutions to assess both its predictive accuracy and its computational efficiency.

To highlight the importance of clustering in the methodology, the ML-based RANS model was compared against a variant trained without clustering, using the same network architecture and hyperparameters without all clustering-related components. The best validation loss for the unclustered model is obtained at epoch 313, with *validation loss* $\approx 1.16 \cdot 10^{-2}$ and *validation MAE* $= 1.96 \cdot 10^{-2}$ in the normalised target space. On the full dataset, the reconstructed physical field reaches $R^2 = 0.9854$, $MSE = 1.3556 \cdot 10^{-1}$ and $MAE = 0.1757$. Although the MSE and MAE are marginally lower than in the clustered case, the lower R^2 indicates that clustering allows the model to better capture the variance of the data and thus to reconstruct the physical field more faithfully. The training and validation loss curves together

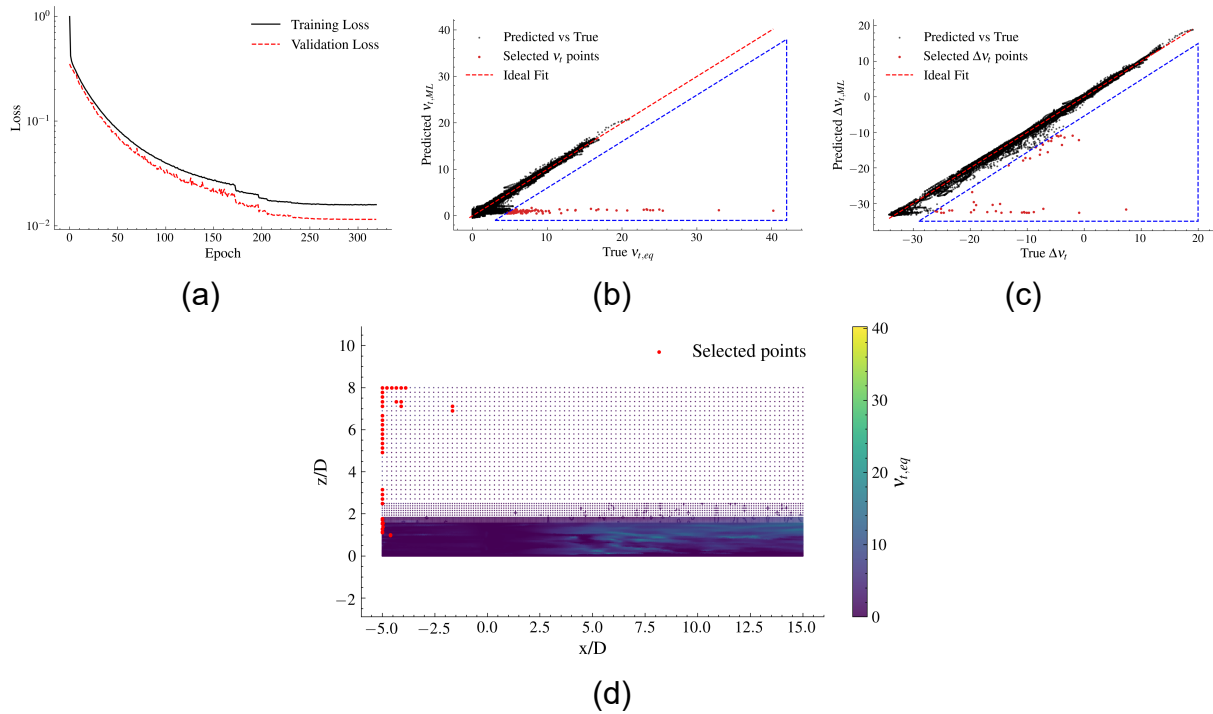


Figure 11. NN results of the model without clustering: (a) training and validation loss curves, (b) scatter plot of predicted vs true v_t values on the test set, (c) scatter plot of predicted vs true Δv_t values on the test set, (d) slice plot of selected high-error points.

with the scatter plots of the predicted and true v_t and Δv_t values on the test set are shown in Fig. 11.

As visible in Figures 11b and 11c, the unclustered model exhibits the same underestimation bias in the inlet region as well as a slightly overfitting trend. More importantly, the absence of clustering leads to a degraded prediction in the near-terrain region, where the flow is strongly influenced by the ground boundary layer and exhibits high spatial variability. In this zone the clustered model produces results that are considerably more consistent with the LES reference, since clustering enables the network to learn separately the distinct flow regimes present in the near-wall and free-stream regions, rather than approximating them with a single global mapping.

Finally, the ML-based RANS model results are shown in Fig. 12.

Velocity deficit along the x-axis at different streamwise distances from the rotor (Fig. 12a) is compared, on grids of different resolution, with the standard RANS model and with the LES reference, together with the predicted velocity, turbulent kinetic energy and eddy viscosity fields (Figures 12b-12d).

On the fine grid (~ 15.9 million cells), the ML-based RANS model consistently outperforms the standard RANS model throughout the wake. In the near-wake region the model shows markedly improved agreement with the LES results, especially in the upper region of the wake where the standard RANS model underestimates the velocity deficit. A moderate underestimation of the velocity deficit persists in the lower near-wake region, attributable to a local underestimation of the v_t . In the far-wake region, after the breakdown of the wake (i.e., from $4D$ in the streamwise direction), the fine-grid model tends to overestimate the velocity deficit with respect to the LES reference, suggesting that at this resolution the v_t correction does not fully reproduce the turbulent mixing responsible for wake recovery.

In terms of computational cost, the ML-based RANS model on the ~ 15.9 million cells grid is about 2.5 times more expensive than the standard LES, which could be regarded as a significant

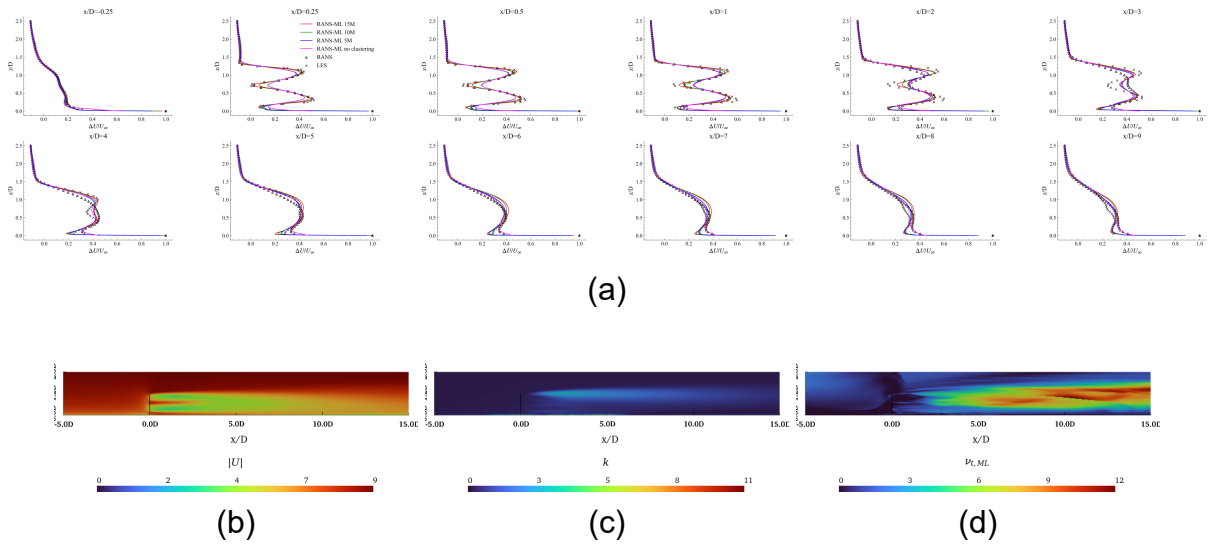


Figure 12. ML-based RANS model results: (a) velocity deficit along the x-axis at different streamwise distances from the rotor, (b) velocity field, (c) k field, (d) $v_{t,ML}$ field.

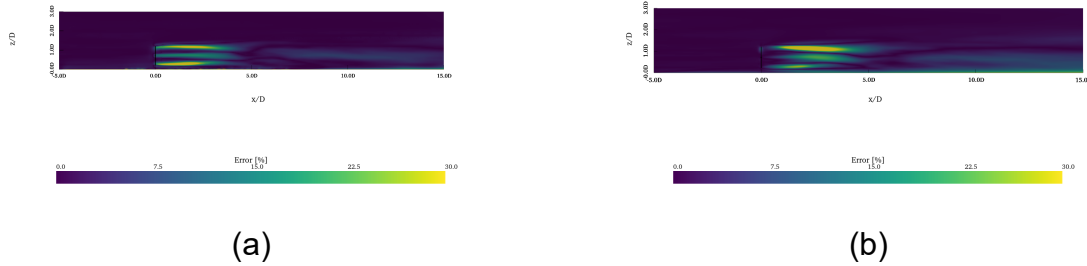


Figure 13. Velocity error field of the ML-based RANS model compared to the standard LES model (a) and of RANS one compared to the standard LES model (b).

drawback. However, the real efficiency of the ML-based RANS model emerges only on the coarser grids.

The most significant, and practically relevant, result is obtained on the ultra-coarse grids. As shown in Fig. 12a, the ML-based RANS model retains its improved near-wake prediction (with lower accuracy with respect to finer grids) while, at the same time, providing a much more accurate description of the far-wake region, including the post-breakdown zone where the fine-grid model overestimates the deficit. This behaviour makes the ultra-coarse configuration the most attractive option for wind farm applications, where an accurate far-wake prediction is critical to estimate the power production of the downstream turbines and to optimise the wind farm layout. The advantage is further reinforced by the reduction in computational cost: on the ~ 4.2 million cells grid the ML-based RANS model is about 1.25 times cheaper than the standard LES, making it a viable tool for large-scale wind farm simulations that require both accuracy and affordability.

Finally, the error in the velocity field is shown in Fig. 13 compared to the standard LES model.

The velocity error fields in Fig. 13 confirm the trends described above. The ML-based

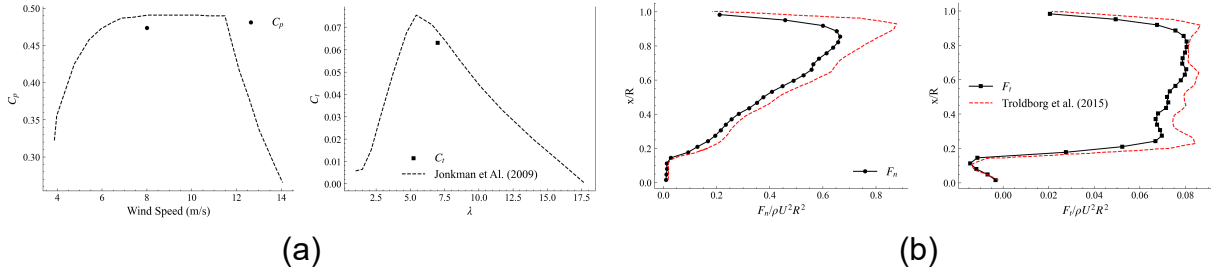


Figure 14. ML-based RANS model results on the predicted forces and power coefficients: (a) C_p and C_t profiles, (b) force profiles.

RANS model (Fig. 13a) achieves a marked reduction of the error in both the near- and far-wake regions with respect to the standard RANS model (Fig. 13b), at the cost of a slight increase in the lower part of the near-wake. Importantly, the error in the wake breakdown region is significantly reduced with respect to the standard RANS model, which directly improves the prediction of the wake recovery and is therefore of primary importance for wind farm applications, since wind turbines are usually located in the far-wake region of the upstream machines.

To assess not only the capability of the model to predict the velocity field but also to investigate its ability to capture the dynamic effects on the wind turbine (and the relative power production), the predicted forces are reported here together with the predicted C_p and C_t values. The predicted forces show good agreement with the reference curves, with a slight underestimation of the overall forces and coefficients. A small deviation in terms of power production can be noted at the C_p corresponding point; this is partly due to the fact that the reference curves refer to an isolated turbine configuration, whereas the ML-based RANS model is applied to a turbine operating in a turbulent ABL and affected by the presence of the ground. Overall, the predicted forces and power coefficients remain in accordance with the reference curves, demonstrating the capability of the ML-based RANS model to predict not only the velocity wake field but also the dynamic effects on the wind turbine and thus its power production, which is a key aspect for wind farm applications.

5 CONCLUSIONS

This work presented a CFD-ML methodology to build a turbulence closure aimed at improving the fidelity of RANS simulations of HAWT wakes while keeping the computational cost compatible with wind farm applications. The ML module is used to define the turbulence model itself, by learning the eddy-viscosity correction required to reproduce high-fidelity LES statistics.

A LES of the NREL-5MW turbine operating in a turbulent atmospheric boundary layer was used to generate the reference data. An LES-equivalent eddy viscosity $\nu_{t,eq}$ was extracted by projecting the deviatoric LES stress onto the Boussinesq form, and a Multi-Layer Perceptron was trained to predict the local correction $\Delta \nu_t = \nu_{t,eq} - \nu_{t,RANS}$ with respect to the RANS baseline. A K-means clustering algorithm was employed to segment the domain into physically distinct flow regions, moving the prediction from a cell-wise to a region-aware basis, and the resulting closure was embedded into a modified OpenFOAM PIMPLE solver to be used in a fully predictive setting. The main findings can be summarised as follows.

- The clustering identified seven physically meaningful flow regions, separating the undisturbed flow from the inlet/ABL-affected, near-wake, mixing/breakdown and far-wake regions, consistently with the underlying flow physics.
- The trained network reproduced the LES-equivalent eddy viscosity with a validation $R^2 =$

0.9934 ($RMSE = 0.2588$, $MAE = 0.1831$ in physical space), confirming that the learned correction drastically reduces the discrepancy with the LES-equivalent closure.

- The comparison with an equivalent network trained without clustering showed that, although the unclustered model attains marginally lower MSE and MAE, its lower R^2 reflects a poorer reconstruction of the data variance. The benefit of clustering is most evident in the near-terrain region, where the clustered model is markedly more consistent with the LES reference, since it is able to learn separately the distinct near-wall and free-stream flow regimes.
- Once embedded in the solver, the ML-based RANS model consistently outperformed the standard RANS model in both the near- and far-wake. On the fine grid the model strongly improved the near-wake prediction, especially in the upper part of the wake, while slightly underestimating the deficit in the lower near-wake and overestimating it in the far-wake after the wake breakdown.
- The most relevant result for practical applications was obtained on the ultra-coarse grids: the model retained its improved near-wake prediction while also accurately reproducing the post-breakdown far-wake recovery, at a computational cost about 1.25 times lower than the reference LES on the $\sim 4.2 \times 10^6$ cells grid. Since downstream turbines are typically located in the far-wake of the upstream machines, this combination of far-wake accuracy and reduced cost is precisely what is required for wind farm layout and energy-yield studies.
- Beyond the velocity field, the model also reproduced the rotor forces and the C_p/C_t coefficients in good agreement with the reference curves, with only a slight underestimation partly attributable to the more realistic ABL and ground-affected operating conditions, demonstrating its ability to capture the aerodynamic loading and the associated power production.

Some limitations should be acknowledged. The model was trained on a single operating condition and on a relatively small dataset, which results in a slight overfitting trend and limits its generalisation. A residual underestimation bias persists in the far-upstream inflow region, inherited from the synthetic inlet boundary condition, and the fine-grid configuration still overestimates the far-wake deficit.

Future work will therefore focus on enlarging the training dataset to include multiple inflow velocities, turbulence intensities and thermal stability conditions, in order to improve the generalisation of the closure, and on extending the methodology to multi-turbine configurations to assess wake-interaction effects. The compactness and the reduced cost of the proposed data-driven closure also make it a promising candidate for future real-time wind farm monitoring and control applications, where accurate yet inexpensive representations of the wake field are essential.

REFERENCES

- [1] Fernando Porté-Agel, Majid Bastankhah, and Sina Shamsoddin. Wind-turbine and wind-farm flows: A review. *Boundary-Layer Meteorology*, 174, 01 2020.
- [2] Kirby S. Heck and Michael F. Howland. Unraveling the effects of atmospheric dynamics on wakes with a controlled synthetic inflow methodology, 2025.

- [3] Mehrshad Gholami Anjiraki, Christian Santoni, Samin Shapourmiandouab, Hossein Seyedzadeh, Jonathan Craig, Fotis Sotiropoulos, and Ali Khosronejad. Wind farm layout optimization using a novel machine learning approach, 09 2025.
- [4] Julia Steiner, Richard Dwight, and Axelle Vire. Data-driven rans closures for wind turbine wakes under neutral conditions. *Computers & Fluids*, 233:105213, 11 2021.
- [5] J. Steiner. *Towards data-driven turbulence modeling for wind turbine wakes*. Dissertation (tu delft), Delft University of Technology, 2023.
- [6] Coleman Moss, Romit Maulik, Patrick Moriarty, and Giacomo Iungo. Predicting wind farm operations with machine learning and the p2d-rans model: A case study for an awaken site. *Wind Energy*, 27:1245–1267, 10 2023.
- [7] David Infield, Matilde Santos Peñas, Davide Astolfi, and Jiarong Hong. Scada data for wind turbine data-driven condition/performance monitoring: A review on state-of-art, challenges and future trends. *Wind Engineering*, 47, 09 2022.
- [8] Kun Yang, Xiaowei Deng, Zilong Ti, Shanghui Yang, Senbin Huang, and Yuhang Wang. A data-driven layout optimization framework of large-scale wind farms based on machine learning. *Renewable Energy*, 218:119240, 2023.
- [9] Karthik Duraisamy, Gianluca Iaccarino, and Heng Xiao. Turbulence modeling in the age of data. *Annual Review of Fluid Mechanics*, 51, 01 2019.
- [10] Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds averaged turbulence modelling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807:155–166, 11 2016.
- [11] R. N. King, C. Adcock, J. Annoni, and K. Dykes. Data-driven machine learning for wind plant flow modeling. *Journal of Physics: Conference Series*, 1037(7):072004, jun 2018.
- [12] Lorenzo Tieghi, Davide Cerbarano, Filippo De Girolamo, Valerio Barnabei, and Giovanni Delibra. A novel methodology for the automatic decomposition of hawt wakes with k-means clustering. *Wind Energy*, 28, 05 2025.
- [13] Andrea Carloni, Giovanni Delibra, Filippo De Girolamo, Valerio Francesco Barnabei, and Lorenzo Tieghi. Data-driven decomposition of horizontal-axis wind-turbine wakes. In *ASME Turbo Expo 2026*, Milan, Italy, 2026.
- [14] J. N. Sorensen and W. Z. Shen. Numerical modeling of wind turbine wakes. *Journal of Fluids Engineering*, 124(2):393–399, 05 2002.
- [15] Pete Bachant, Anders Goude, and Martin Wosnik. turbinesfoam. Zenodo, 04 2016.
- [16] F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.
- [17] Franck Nicoud and Frédéric Ducros. Subgrid-scale stress modelling based on the square of the velocity gradient tensor. *Flow, turbulence and Combustion*, 62(3):183–200, 1999.
- [18] J. C. Kaimal, J. C. Wyngaard, Y. Izumi, and O. R. Coté. Spectral characteristics of surface-layer turbulence. *Quarterly Journal of the Royal Meteorological Society*, 98(417):563–589, 1972.

- [19] Lars Davidson. Using isotropic synthetic fluctuations as inlet boundary conditions for unsteady simulations. *Advances and Applications in Fluid Mechanics*, 1:1–35, 2007.
- [20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [21] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [22] Jason Jonkman. Definition of a 5-mw reference wind turbine for offshore system development, 2009.
- [23] Luis Martínez Tossas, Stefano Leonardi, Matthew Churchfield, and Patrick Moriarty. A comparison of actuator disk and actuator line wind turbine models and best practices for their use. 01 2012.
- [24] Lorenzo Cappugi, Alessio Castorrini, Aldo Bonfiglioli, Edmondo Minisci, and Sergio Campobasso. Machine learning-enabled prediction of wind turbine energy yield losses due to general blade leading edge erosion. *Energy Conversion and Management*, 245:114567, 10 2021.
- [25] Niels Trolborg, Frederik Zahle, Pierre-Elouan Réthoré, and Niels N. Sørensen. Comparison of wind turbine wake properties in non-sheared inflow predicted by different computational fluid dynamics rotor models. *Wind Energy*, 18(7):1239–1250, 2015.